ALFIDO TECH
Internship Program

TASK REPORT

Deploying a Docker Container on a Cloud Virtual Machine (AWS EC2)

Intern Name	Habiba Shafait
Program	BS Information Technology
Internship	Alfido Technologies
Task Title	Deploy a Docker Container on a Cloud VM
Cloud Platform	Amazon Web Services (AWS) – EC2
Operating System	Ubuntu (EC2 instance)
Tool Used	Docker
Date	July 2026

1. Objective

The objective of this task was to gain hands-on experience with cloud computing and containerization by launching a virtual machine (VM) on Amazon Web Services (AWS), installing Docker on it, and deploying a live web server inside a Docker container. The task demonstrates the complete workflow of provisioning cloud infrastructure, configuring a Linux environment, and running a containerized application accessible over the public internet.

2. Tools & Technologies Used

- Amazon Web Services (AWS) – EC2 (Elastic Compute Cloud)
- Ubuntu Server (Linux operating system running on the cloud VM)
- EC2 Instance Connect (browser-based SSH terminal)
- Docker Engine (docker.io package)
- Nginx (official Docker image used as the sample web server)
- Linux shell commands (apt, systemctl, docker CLI)

3. Step-by-Step Procedure

STEP 1: Launch an EC2 Virtual Machine on AWS

An Ubuntu-based EC2 instance named "MyWebServer" was launched from the AWS Management Console using a t3.micro instance type. The AWS EC2 dashboard confirms that the instance is in the "Running" state with its status checks initializing, as shown below.

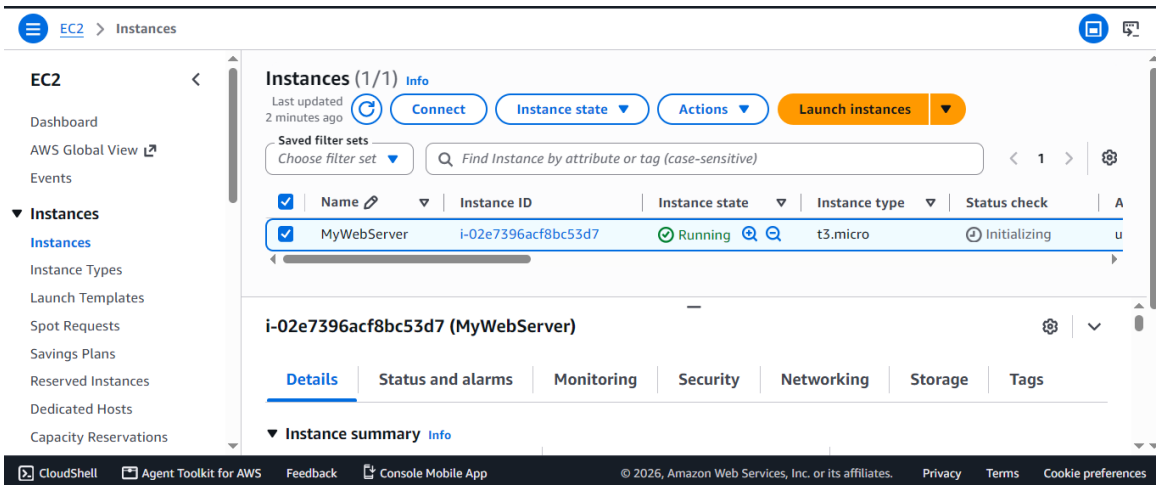

Fig 1: EC2 instance "MyWebServer" running in the AWS console

STEP 2: Connect to the VM and Update the System

Once the instance was running, it was accessed through EC2 Instance Connect, which opens a secure browser-based SSH terminal directly into the VM (logged in as the default ubuntu user). Before installing any software, the package index was refreshed using the following command so that the latest versions of all packages would be available:

```
sudo apt update -y
```

The terminal output below shows the package lists being fetched successfully from Ubuntu's repositories.

```
Last login: Fri Jul 17 08:12:49 2026 from 18.206.107.27
ubuntu@ip-172-31-18-31:~$ sudo apt update -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 Packages [358 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [91.7 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [53.1 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/universe amd64v3 Packages [209 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [65.0 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [153 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [31.2 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.1 kB]
Fetched 1280 kB in 1s (1820 kB/s)
70 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-18-31:~$
```

Fig 2: Connecting via SSH and updating the package index

STEP 3: Install Docker on the VM

Docker was then installed on the Ubuntu VM using the official Ubuntu package (docker.io):

```
sudo apt install docker.io -y
```

APT resolved and installed Docker along with its required dependencies, as shown in the terminal output.

```
70 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-18-31:~$ sudo apt install docker.io -y
Installing:
  docker.io

Installing dependencies:
  bridge-utils  containerd  dns-root-data  dnsmasq-base  pigz  runc  ubuntu-fan

Suggested packages:
  ifupdown    cgroupfs-mount  debootstrap    docker-compose-v2  rinse        zfs-fuse
  aufs-tools  | cgroup-lite   docker-buildx  docker-doc         rootlesskit  | zfsutils
```

Fig 3: Installing Docker using apt

STEP 4: Start, Enable, and Verify Docker

After installation, the Docker service was started and enabled so that it would automatically run on every system boot:

```
sudo systemctl start docker
sudo systemctl enable docker
docker --version
```

The docker --version command confirmed that Docker Engine had been installed and was ready to use.

```
ubuntu@ip-172-31-18-31:~$ sudo systemctl start docker
ubuntu@ip-172-31-18-31:~$ sudo systemctl enable docker
ubuntu@ip-172-31-18-31:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-18-31:~$ sudo docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-18-31:~$
```

i-02e7396acf8bc53d7 (MyWebServer)

PublicIPs: 54.167.115.79 PrivateIPs: 172.31.18.31

Fig 4: Starting/enabling the Docker service and confirming the installed version

STEP 5: Run a Docker Container (Nginx Web Server)

A container running the official nginx image was then launched, mapping port 80 of the container to port 80 of the host VM so the web server would be reachable from the internet:

```
sudo docker rm my-web-server
sudo docker run -d -p 80:80 --name my-web-server nginx
sudo docker ps
```

The docker run command downloaded/started the container and returned its container ID. The docker ps command was then used to confirm that the container named my-web-server was up and running, with port 80 correctly published.

Fig 5: Running the Nginx container and verifying it with docker ps

STEP 6: Access the Web Server Using the Public IP

With the container running, the EC2 instance's public IP address (54.167.115.79) was entered into a web browser to test connectivity to the containerized application over the internet.

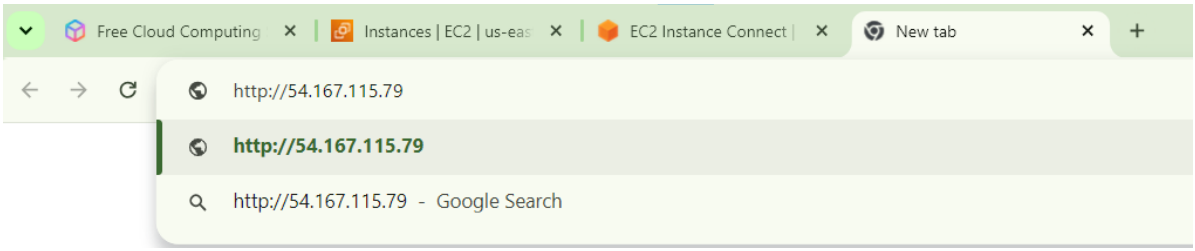

Fig 6: Navigating to the EC2 instance's public IP address in the browser

STEP 7: Verify Successful Deployment

The browser successfully loaded the default "Welcome to nginx!" page, confirming that the Docker container was deployed correctly and is accessible from outside the cloud VM, over the public internet.

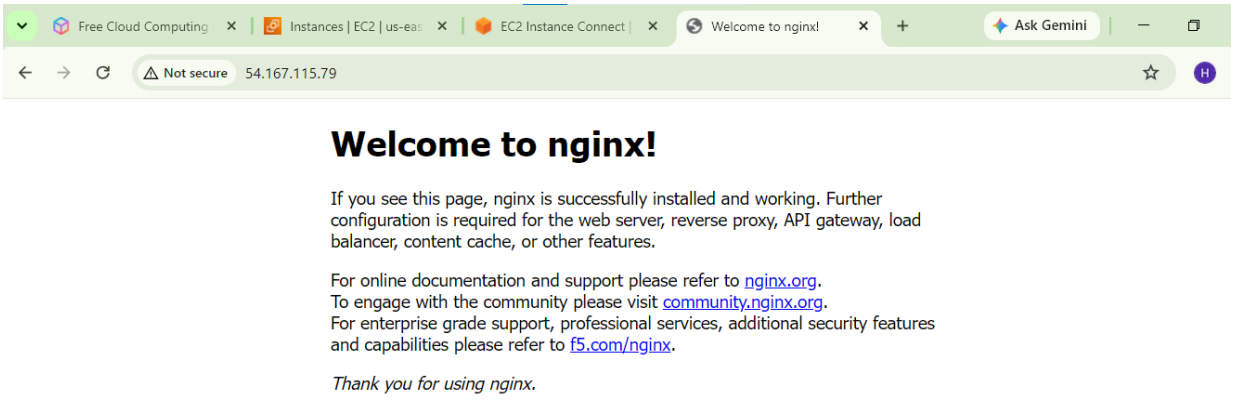

Fig 7: Nginx welcome page confirming successful deployment

4. Result

The task was completed successfully. A cloud-based virtual machine was provisioned on AWS EC2, Docker was installed and configured on it, and a containerized Nginx web server was deployed and made publicly accessible through the instance's public IP address.

5. Conclusion

This task provided practical experience in combining cloud computing with containerization — two core skills in modern IT and DevOps workflows. Key learnings included:

- Provisioning and managing a virtual machine on a public cloud platform (AWS EC2).
- Connecting to a remote Linux server securely via SSH / EC2 Instance Connect.
- Installing and configuring the Docker Engine on a Linux system.
- Building and running containers, and mapping container ports to host ports.
- Verifying a deployment end-to-end by accessing the application over the public internet.

These skills are directly relevant to deploying, scaling, and securing backend services and web applications — a valuable foundation for future work in IT infrastructure, DevOps, and application deployment, including in security-conscious environments such as banking and financial technology systems.